

Optimizador de Rutas de Recolección

Guía técnica del código, explicado paso a paso para aprender

Este documento explica, línea por línea cuando hace falta, cómo funciona la aplicación `app.py` + `db.py`: desde cómo se guardan los datos, cómo se consultan distancias reales por carretera, cómo se resuelve el problema de la ruta óptima con OR-Tools, hasta cómo se dibuja el mapa final.

Nota: Está pensado para alguien que sabe programar lo básico en Python pero nunca usó Streamlit, OR-Tools, ni consumió una API externa. Cada sección explica primero el qué, después el por qué, y al final el código real del proyecto.

Contenido

1. Visión general del proyecto
2. Arquitectura: dos archivos, dos responsabilidades
3. db.py — cómo se guardan los datos
 - 3.1 El patrón context manager (get_conn)
 - 3.2 init_db — crear las tablas
 - 3.3 El mapeo de nombres de columna
 - 3.4 cargar_puntos y guardar_puntos
 - 3.5 Configuración persistente (obtener_config / guardar_config)
4. app.py — funciones auxiliares explicadas una por una
 - 4.1 haversine — distancia en línea recta
 - 4.2 geocodificar_direccion — texto a coordenadas
 - 4.3 obtener_matriz_osrm — distancias reales por carretera
 - 4.4 obtener_ruta_osrm — el dibujo de cada tramo
 - 4.5 resolver_vrp — el corazón matemático (OR-Tools)
 - 4.6 generar_link_google_maps
5. app.py — la interfaz, de arriba hacia abajo
 - 5.1 Estado de sesión y carga inicial de datos
 - 5.2 La barra lateral (sidebar)
 - 5.3 La tabla editable de puntos
 - 5.4 El botón "Calcular Ruta Óptima": el flujo completo
 - 5.5 Mostrar resultados: métricas, tabla y mapa
6. Conceptos clave para seguir aprendiendo
7. Glosario rápido

1. Visión general del proyecto

La aplicación resuelve un problema muy concreto: dado un depósito (bodega) y una lista de puntos a visitar, ¿en qué orden conviene visitarlos para recorrer la menor distancia posible, y a qué hora se llega a cada uno? Esto se conoce como un problema de ruteo de vehículos (VRP, por Vehicle Routing Problem), y en su versión más simple — un solo vehículo — se llama TSP (Travelling Salesman Problem).

El flujo completo, de principio a fin, es siempre el mismo:

- 1 El usuario carga puntos (a mano o con direcciones) en una tabla editable.
- 2 Se calcula una matriz de distancias entre todos los puntos, usando un servicio externo de mapas (OSRM) para que sean distancias reales por carretera, no en línea recta.
- 3 Esa matriz se le pasa a OR-Tools, una librería de Google especializada en este tipo de problemas, que devuelve el orden óptimo de visita.
- 4 Con ese orden, se le vuelve a preguntar a OSRM por la geometría real de cada tramo (para dibujar el camino en el mapa, no solo una línea recta entre puntos).
- 5 Se calculan horarios estimados y peso acumulado, sumando tiempos de viaje y de parada.
- 6 Se muestra todo: métricas, una tabla detallada, un mapa interactivo y botones para exportar a CSV o abrir la ruta en Google Maps.

Nota: Ninguno de estos pasos es "magia": son funciones de Python normales, una llamando a la siguiente. Este documento las recorre en el mismo orden en que el código las ejecuta.

2. Arquitectura: dos archivos, dos responsabilidades

El proyecto está dividido en dos archivos, y esa división no es casual: separa qué se muestra en pantalla de cómo se guardan los datos. Es un principio general de buen diseño de software llamado separación de responsabilidades.

db.py — la capa de persistencia

Solo sabe hablar con la base de datos SQLite. No conoce Streamlit, no dibuja nada, no sabe qué es un mapa. Si el día de mañana cambiás SQLite por otra base de datos, en teoría solo tenés que tocar este archivo.

app.py — la capa de interfaz y orquestación

Dibuja la página, lee lo que el usuario escribe, llama a db.py para guardar/leer, llama a las funciones de OSRM y OR-Tools, y junta todos los resultados para mostrarlos.

Nota: Esta separación es justamente lo que te permite, a futuro, cambiar la lógica de optimización (por ejemplo a varios camiones) sin tener que tocar cómo se guardan los puntos en la base de datos.

3. db.py — cómo se guardan los datos

Streamlit, por defecto, no recuerda nada entre una recarga de página y la siguiente: cada vez que interactuás con la app, el script se vuelve a ejecutar de arriba a abajo. Sin una base de datos, cada clic borraría lo que cargaste. db.py existe para resolver exactamente ese problema, usando SQLite: una base de datos liviana que vive en un solo archivo (rutas.db), sin necesidad de instalar ni configurar un servidor de base de datos aparte.

3.1 El patrón context manager (get_conn)

Antes de tocar la base de datos hay que abrir una conexión, y después de usarla hay que cerrarla — siempre, incluso si algo falla en el medio. Olvidarse de cerrar conexiones es una fuente clásica de bugs. Python resuelve esto con el patrón context manager, que se usa con la palabra clave with:

db.py — get_conn()

```
@contextmanager
def get_conn():
    conn = sqlite3.connect(DB_PATH)
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()
```

El decorador @contextmanager convierte esta función en algo que se puede usar como with get_conn() as conn: Todo lo que pasa antes del yield se ejecuta al entrar al bloque with; todo lo que está después se ejecuta al salir del bloque, ya sea que todo haya ido bien o que haya ocurrido un error. El finally garantiza que conn.close() se ejecute siempre, y el commit() guarda los cambios solo si no hubo excepciones (porque está fuera del try... en realidad acá está dentro del try, así que si algo falla, ese commit no se ejecuta y los cambios no se guardan a medias).

3.2 init_db — crear las tablas

Esta función se llama una sola vez, al arrancar la app. Usa CREATE TABLE IF NOT EXISTS, que es una instrucción "segura de repetir": si la tabla ya existe, no hace nada; si no existe, la crea. Así podés llamar a init_db() en cada recarga sin miedo a borrar nada.

db.py — init_db()

```
def init_db():
    with get_conn() as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS puntos (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                nombre TEXT NOT NULL,
                direccion TEXT,
                latitud REAL,
                longitud REAL,
                peso_kg REAL DEFAULT 0,
                camion_asignado TEXT
            )
        """)
        conn.execute("""
            CREATE TABLE IF NOT EXISTS config (
                clave TEXT PRIMARY KEY,
                valor TEXT
            )
        """)
```

Dos tablas, dos propósitos distintos:

- puntos: una fila por cada parada de recolección. La columna camion_asignado está vacía por ahora — es un "hueco" dejado a propósito para cuando se agreguen varios camiones (Fase 2), así no hay que migrar la tabla.
- config: una tabla genérica de clave-valor (como un diccionario guardado en disco). Sirve para cualquier ajuste — hora de inicio, velocidad, capacidad — sin tener que crear una columna nueva por cada configuración que se agregue en el futuro.

3.3 El mapeo de nombres de columna

La tabla editable que ve el usuario tiene columnas en español, con tildes y paréntesis: "Nombre", "Dirección", "Peso (kg)". La base de datos, en cambio, usa nombres simples en snake_case: nombre, direccion, peso_kg. Esto no es un descuido — es buena práctica: el nombre de una columna de base de datos no debería depender de cómo se decide mostrar esa información en una interfaz que puede cambiar de idioma o de formato. El diccionario que conecta ambos mundos es:

db.py — diccionarios de mapeo

```
DB_TO_UI = {
    "nombre": "Nombre",
    "direccion": "Dirección",
    "latitud": "Latitud",
    "longitud": "Longitud",
    "peso_kg": "Peso (kg)",
}
UI_TO_DB = {v: k for k, v in DB_TO_UI.items()}
```

La segunda línea es un truco muy común en Python: una comprensión de diccionario que invierte claves y valores. Así no hace falta escribir el mapeo dos veces a mano (con el riesgo

de que se desincronicen).

3.4 cargar_puntos y guardar_puntos

db.py — cargar_puntos()

```
def cargar_puntos():
    with get_conn() as conn:
        df = pd.read_sql_query(
            "SELECT nombre, direccion, latitud, longitud, peso_kg "
            "FROM puntos ORDER BY id",
            conn,
        )
    return df.rename(columns=DB_TO_UI)
```

`pd.read_sql_query` ejecuta el SQL y devuelve directamente un DataFrame de pandas — no hace falta recorrer filas a mano. `ORDER BY id` asegura que los puntos siempre se muestren en el mismo orden en que se guardaron. Al final, `.rename(columns=DB_TO_UI)` traduce los nombres de columna al español antes de devolver la tabla, lista para mostrarse en el `data_editor` de Streamlit.

db.py — guardar_puntos()

```
def guardar_puntos(df_ui):
    df = df_ui.rename(columns=UI_TO_DB).copy()
    for col in ["nombre", "direccion", "latitud", "longitud", "peso_kg"]:
        if col not in df.columns:
            df[col] = None
    df = df[["nombre", "direccion", "latitud", "longitud", "peso_kg"]]
    df = df.dropna(subset=["nombre"])

    with get_conn() as conn:
        conn.execute("DELETE FROM puntos")
        if len(df) > 0:
            df.to_sql("puntos", conn, if_exists="append", index=False)
```

Acá pasan varias cosas interesantes, en orden:

- 1 Se traducen las columnas de español a `snake_case` con el diccionario inverso.
- 2 Se asegura que existan las cinco columnas esperadas, aunque el usuario no haya tocado la columna "Dirección" (si no existe, se crea vacía con `None`).
- 3 Se reordenan las columnas siempre en el mismo orden — importante porque `to_sql` inserta por posición.
- 4 `dropna(subset=["nombre"])` descarta filas vacías que deja el editor de Streamlit cuando el usuario hace clic en "+" para agregar una fila y no la llena.
- 5 Se borra todo el contenido de la tabla y se vuelve a insertar desde cero.

Nota: El último paso ("borrar todo e insertar de nuevo") es la estrategia más simple posible para guardar una tabla completa, y funciona perfectamente bien para decenas o cientos de puntos. No es la más eficiente para miles de filas (ahí convendría comparar fila por fila y solo actualizar lo que cambió), pero para este caso de uso es la opción correcta: simple, predecible, y sin casos raros que depurar.

3.5 Configuración persistente

La tabla config guarda pares clave-valor. Como SQLite no distingue tipos estrictos en esta tabla (todo se guarda como texto), quien lee el valor es responsable de convertirlo de nuevo a número cuando haga falta — eso se ve en app.py con cosas como `int(db.obtener_config("velocidad_kmh", 40))`.

db.py — obtener_config() y guardar_config()

```
def obtener_config(clave, default=None):
    with get_conn() as conn:
        cur = conn.execute("SELECT valor FROM config WHERE clave = ?", (clave,))
        row = cur.fetchone()
        return row[0] if row is not None else default

def guardar_config(clave, valor):
    with get_conn() as conn:
        conn.execute(
            """INSERT INTO config (clave, valor) VALUES (?, ?)
            ON CONFLICT(clave) DO UPDATE SET valor = excluded.valor""",
            (clave, str(valor)),
        )
```

Dos detalles importantes de seguridad y de SQL:

- Los signos ? dentro del SQL, con los valores pasados aparte en una tupla (clave,), son parámetros preparados. Nunca se debe construir un SQL concatenando texto del usuario directamente (eso abre la puerta a inyección SQL); los ? evitan ese problema dejando que la librería escape los valores.
- `ON CONFLICT(clave) DO UPDATE SET ...` es un upsert: inserta la fila si la clave no existe, y la actualiza si ya existía. Evita tener que hacer primero un `SELECT` para decidir entre `INSERT` o `UPDATE`.

4. app.py — funciones auxiliares, explicadas una por una

Estas son funciones "puras" en su mayoría: reciben datos, devuelven datos, y no dibujan nada en pantalla directamente. Es buena práctica separar así la lógica de cálculo de la lógica de interfaz, porque hace que estas funciones se puedan probar y entender de forma aislada.

4.1 haversine — distancia en línea recta

Calcula la distancia entre dos puntos GPS como si no hubiera calles ni obstáculos — una línea recta sobre la superficie de la Tierra (que no es plana, así que no alcanza con el teorema de Pitágoras). Se usa como respaldo cuando OSRM no está disponible.

app.py — haversine()

```
def haversine(c1, c2):
    R = 6_371_000
    lat1, lon1 = math.radians(c1[0]), math.radians(c1[1])
    lat2, lon2 = math.radians(c2[0]), math.radians(c2[1])
    a = math.sin((lat2 - lat1) / 2) ** 2 + math.cos(lat1) * math.cos(lat2) * \
        math.sin((lon2 - lon1) / 2) ** 2
    return int(R * 2 * math.asin(math.sqrt(a)))
```

Es la implementación de la fórmula de Haversine, el método estándar para calcular distancias "en línea recta" sobre una esfera. $R = 6\,371\,000$ es el radio promedio de la Tierra en metros. La función convierte las coordenadas de grados a radianes (porque las funciones trigonométricas de Python trabajan en radianes), aplica la fórmula, y devuelve la distancia en metros como número entero.

Nota: No necesitás memorizar esta fórmula para entender el resto del programa — alcanza con saber que recibe dos coordenadas (lat, lon) y devuelve metros.

4.2 geocodificar_direccion — texto a coordenadas

Convierte una dirección escrita en texto ("San José, Costa Rica") en coordenadas GPS, consultando un servicio gratuito llamado Nominatim (el motor de búsqueda de OpenStreetMap).

app.py — geocodificar_direccion()

```
def geocodificar_direccion(direccion):
    url = "https://nominatim.openstreetmap.org/search"
    params = {"q": direccion, "format": "json", "limit": 1}
    headers = {"User-Agent": "optimizador-rutas-app/1.0"}
    try:
        resp = requests.get(url, params=params, headers=headers, timeout=10)
        resp.raise_for_status()
        data = resp.json()
        if data:
            return float(data[0]["lat"]), float(data[0]["lon"]), None
        return None, None, "no se encontraron resultados"
    except requests.exceptions.Timeout:
        return None, None, "timeout consultando Nominatim"
    except requests.exceptions.ConnectionError:
        return None, None, "sin conexion a Nominatim"
    except Exception as e:
        return None, None, f"error inesperado ({e})"
```

Algunos detalles que vale la pena notar:

- `params` es un diccionario que `requests` convierte automáticamente en parámetros de URL (`?q=...&format=json&limit=1`). Es más legible que armar la URL a mano con f-strings.
- El `User-Agent` personalizado es un requisito de buena ciudadanía con la API de Nominatim: sin identificarte, el servicio puede bloquear tus solicitudes.
- La función devuelve siempre una tupla de tres valores: (latitud, longitud, error). Cuando todo va bien, `error` es `None`; cuando falla, las coordenadas son `None` y el tercer valor explica por qué. Este patrón — devolver el resultado y el motivo del error en la misma tupla — evita tener que usar excepciones para controlar el flujo normal del programa, y le permite a quien llama a la función decidir qué hacer con el error sin necesidad de un `try/except` adicional.
- Cada tipo de fallo de red (`Timeout`, `ConnectionError`) se captura por separado, así el mensaje que ve el usuario es específico en vez de un genérico "algo salió mal".

4.3 obtener_matriz_osrm — distancias reales por carretera

OR-Tools necesita saber la distancia entre cada par posible de puntos antes de poder decidir el mejor orden de visita. Esta función construye esa tabla completa (la "matriz de distancias") consultando una sola vez al servicio OSRM (Open Source Routing Machine), que calcula distancias siguiendo calles reales.

app.py — obtener_matriz_osrm()

```
def obtener_matriz_osrm(locations):
    coords_str = ";".join(f"{lon},{lat}" for lat, lon in locations)
    url = f"http://router.project-osrm.org/table/v1/driving/{coords_str}?annotations=distance"
    try:
        resp = requests.get(url, timeout=20)
        resp.raise_for_status()
        data = resp.json()
        if data.get("code") == "Ok":
            return [[int(d) for d in row] for row in data["distances"]], True, None
        error_msg = f"OSRM respondió con código '{data.get('code')}'"
    except requests.exceptions.Timeout:
        error_msg = "OSRM no respondió a tiempo (timeout)."
    except requests.exceptions.ConnectionError:
        error_msg = "No se pudo conectar con OSRM."
    except Exception as e:
        error_msg = f"Error inesperado consultando OSRM: {e}"

    n = len(locations)
    matriz = [[0 if i == j else haversine(locations[i], locations[j])
               for j in range(n)] for i in range(n)]
    return matriz, False, error_msg
```

Por partes:

- `coords_str` arma la lista de coordenadas en el formato exacto que pide la API de OSRM: longitud,latitud separadas por punto y coma (notar que OSRM espera longitud primero, al revés de como normalmente pensamos "lat, lon" — una fuente común de errores si no se presta atención).
- El endpoint `/table/` de OSRM, en una sola llamada HTTP, devuelve la distancia entre todos los pares de puntos a la vez — mucho más eficiente que pedir las distancias de a una.
- `data["distances"]` es una matriz (lista de listas) donde la posición `[i][j]` es la distancia en metros desde el punto `i` hasta el punto `j`.
- Si la llamada falla por cualquier motivo, el código no se detiene: cae al plan B y construye la misma matriz usando `haversine()` punto por punto. Esto es lo que en ingeniería de software se llama degradación elegante (*graceful degradation*): la app sigue funcionando, aunque con resultados menos precisos, en vez de romperse por completo.
- La función devuelve tres cosas: la matriz, un booleano que indica si se usó OSRM de verdad o el respaldo, y el mensaje de error si lo hubo (o `None` si todo salió bien).

4.4 obtener_ruta_osrm — el dibujo de cada tramo

Una vez que OR-Tools decide el orden de visita, esta función le pregunta a OSRM, tramo por tramo, por dónde pasa exactamente la carretera entre dos puntos consecutivos — no solo la distancia, sino la forma de la línea para dibujarla en el mapa.

app.py — obtener_ruta_osrm()

```
def obtener_ruta_osrm(origen, destino):
    url = (
        f"http://router.project-osrm.org/route/v1/driving/"
        f"{origen[1]},{origen[0]};{destino[1]},{destino[0]}"
        f"?overview=full&geometries=geojson"
    )
    try:
        resp = requests.get(url, timeout=15)
        resp.raise_for_status()
        data = resp.json()
        if data.get("code") == "Ok":
            coords = data["routes"][0]["geometry"]["coordinates"]
            dist_m = data["routes"][0]["legs"][0]["distance"]
            return [(lat, lon) for lon, lat in coords], dist_m, None
            return [origen, destino], haversine(origen, destino), f"OSRM codigo '{data.get('code')}'"
    except requests.exceptions.Timeout:
        return [origen, destino], haversine(origen, destino), "timeout"
    except requests.exceptions.ConnectionError:
        return [origen, destino], haversine(origen, destino), "sin conexion"
    except Exception as e:
        return [origen, destino], haversine(origen, destino), f"error inesperado ({e})"
```

El endpoint `/route/` (distinto del `/table/` de la función anterior) devuelve la geometría completa del camino como una lista de coordenadas (geojson), que después se invierten de `[lon, lat]` a `(lat, lon)` porque así es como las espera Folium, la librería que dibuja el mapa. Si algo falla, el respaldo dibuja simplemente una línea recta entre origen y destino — la ruta sigue siendo usable, solo que ese tramo en particular no sigue la forma real de la calle.

Nota: Esta función se llama una vez por cada tramo de la ruta (no una sola vez para toda la ruta). Si tenés 10 paradas, son 10 llamadas HTTP separadas. Es una de las mejoras pendientes mencionadas en la conversación: el endpoint de OSRM acepta varios puntos en una sola llamada, lo que ahorraría tiempo de espera.

4.5 resolver_vrp — el corazón matemático (OR-Tools)

Esta es la función más conceptualmente densa del proyecto. Usa OR-Tools, una librería de optimización de Google, para resolver el problema de encontrar el orden de visita que minimiza la distancia total recorrida.

app.py — resolver_vrp()

```
def resolver_vrp(distancias):
    manager = pywrapcp.RoutingIndexManager(len(distancias), 1, 0)
    routing = pywrapcp.RoutingModel(manager)

    def cb(from_idx, to_idx):
        return distancias[manager.IndexToNode(from_idx)][manager.IndexToNode(to_idx)]

    t = routing.RegisterTransitCallback(cb)
    routing.SetArcCostEvaluatorOfAllVehicles(t)

    params = pywrapcp.DefaultRoutingSearchParameters()
    params.first_solution_strategy = \
        routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST_ARC
    params.local_search_metaheuristic = \
        routing_enums_pb2.LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH
    params.time_limit.seconds = 10

    sol = routing.SolveWithParameters(params)
    if not sol:
        return None

    idx = routing.Start(0)
    ruta = []
    while not routing.IsEnd(idx):
        ruta.append(manager.IndexToNode(idx))
        idx = sol.Value(routing.NextVar(idx))
    ruta.append(manager.IndexToNode(idx))
    return ruta
```

Vamos paso a paso, porque cada línea hace algo específico:

RoutingIndexManager y RoutingModel

`RoutingIndexManager(len(distancias), 1, 0)` crea el "administrador" del problema, con tres datos: cuántos nodos hay en total (puntos + depósito), cuántos vehículos se van a usar (aquí, 1 — esto es justamente lo que cambiará en la Fase 2 cuando se agreguen más camiones), y cuál nodo es el depósito de inicio/fin (el índice 0, que en el código siempre corresponde al depot porque así se construye la lista `LOCATIONS` más abajo en el archivo). `RoutingModel` es el objeto principal sobre el que se define y resuelve el problema.

El callback de costos (cb)

OR-Tools no sabe nada de geografía: solo entiende "nodos" identificados con números internos. `RegisterTransitCallback` le enseña cómo calcular el costo de ir de un nodo a otro, traduciendo esos índices internos (`from_idx`, `to_idx`) a las posiciones reales en la matriz de distancias mediante `manager.IndexToNode(...)`. `SetArcCostEvaluatorOfAllVehicles(t)` le dice al modelo: "usá esta función para saber qué tan caro es cada tramo".

Estrategia de búsqueda

Encontrar la solución perfecta a un TSP es computacionalmente muy costoso a medida que crecen los puntos (el número de rutas posibles crece factorialmente). En la práctica, se usan heurísticas: estrategias que encuentran una solución muy buena (no necesariamente la

matemáticamente óptima) en un tiempo razonable.

- `PATH_CHEAPEST_ARC` es la estrategia para construir una primera solución rápida: en cada paso, elige ir al punto más cercano todavía no visitado (parecido a la intuición de "el vecino más cercano").
- `GUIDED_LOCAL_SEARCH` toma esa primera solución y la va mejorando iterativamente, probando pequeños cambios (intercambiar el orden de dos paradas, mover una parada a otra posición) y quedándose con las mejoras, evitando quedar atascado en una solución mediocre.
- `time_limit.seconds = 10` le pone un límite de tiempo a esa mejora iterativa: después de 10 segundos, devuelve la mejor solución encontrada hasta ese momento, sea o no la óptima absoluta.

Extraer la ruta de la solución

Una vez resuelto, la solución no viene como una lista lista para usar — hay que recorrerla nodo por nodo. `routing.Start(0)` da el punto de partida del vehículo número 0 (el único que hay). El bucle `while not routing.IsEnd(idx)` va siguiendo la cadena de "siguiente nodo" (`sol.Value(routing.NextVar(idx))`) hasta volver al depósito, acumulando cada nodo visitado en la lista ruta. El resultado final es simplemente una lista de números: el orden en que conviene visitar los índices de `LOCATIONS`.

Nota: Si OR-Tools no logra encontrar ninguna solución (por ejemplo, si las restricciones son imposibles de cumplir), `SolveWithParameters` devuelve un valor "falsy", y la función devuelve `None` — que `app.py` revisa más adelante con `if not ruta_nodos:` para mostrar un error en vez de romperse.

4.6 generar_link_google_maps

`app.py` — `generar_link_google_maps()`

```
def generar_link_google_maps(locations_in_order):
    origin = locations_in_order[0]
    destination = locations_in_order[-1]
    waypoints = locations_in_order[1:-1]

    url = (
        "https://www.google.com/maps/dir/?api=1"
        f"&origin={origin[0]},{origin[1]}"
        f"&destination={destination[0]},{destination[1]}"
    )
    if waypoints:
        wp_str = "|".join(f"{lat},{lon}" for lat, lon in waypoints[:10])
        url += f"&waypoints={wp_str}"
    url += "&travelmode=driving"
    return url, len(waypoints) > 10
```

Google Maps tiene su propio formato de URL para abrir una ruta con varias paradas: origin, destination, y una lista de waypoints separados por el carácter |. La función toma la lista de

coordenadas ya ordenada por OR-Tools y la traduce a ese formato. `locations_in_order[1:-1]` es slicing de Python: toma todos los elementos excepto el primero y el último (que son el origen y el destino, ya separados aparte). El segundo valor que devuelve la función (`len(waypoints) > 10`) avisa si hay más paradas de las que Google Maps admite sin una API de pago, para poder mostrar una advertencia en la interfaz.

5. app.py — la interfaz, de arriba hacia abajo

A diferencia de un programa tradicional, un script de Streamlit se ejecuta de nuevo, completo, cada vez que el usuario interactúa con algo (escribe en un campo, hace clic en un botón). Entender esto es la clave para entender todo lo que sigue: no hay un "bucle de eventos" como en otros frameworks de interfaz — hay un script que corre de arriba a abajo, una y otra vez.

5.1 Estado de sesión y carga inicial de datos

Si el script se re-ejecuta entero en cada interacción, ¿cómo hace para no perder el resultado de un cálculo que ya hizo? Streamlit ofrece `st.session_state`: un diccionario especial que sí sobrevive entre re-ejecuciones, mientras dure la sesión del navegador.

app.py — inicialización de `session_state`

```
if "resultados" not in st.session_state:
    st.session_state.resultados = None
```

Esto se lee como: "si todavía no calculé nada en esta sesión, arrancá con None". Más adelante, cuando se presiona el botón de calcular, el resultado se guarda ahí (`st.session_state.resultados = {...}`), y la sección que dibuja el mapa lo lee de ahí en vez de recalcularlo todo en cada recarga.

Justo después, la app decide qué datos mostrar en la tabla de puntos: si ya hay datos guardados en SQLite, los carga; si es la primera vez que se usa la app, siembra la base de datos con seis puntos de ejemplo.

app.py — carga inicial

```
if db.hay_puntos_guardados():
    datos_iniciales = db.cargar_puntos()
else:
    datos_iniciales = datos_default
    db.guardar_puntos(datos_default)
```

5.2 La barra lateral (sidebar)

El bloque `with st.sidebar:` agrupa todos los controles que aparecen en el panel izquierdo. Cada campo usa el patrón `value=db.obtener_config("clave", default)`: al recargar la página, el campo no vuelve a su valor por defecto, sino al último valor guardado.

app.py — ejemplo de campo con valor persistido

```
velocidad_kmh = st.number_input(
    "Velocidad promedio (km/h)", min_value=10, max_value=120,
    value=int(db.obtener_config("velocidad_kmh", 40)),
)
```


Notar el `int(...)` envolviendo la llamada: como la tabla `config` guarda todo como texto, hay que convertir explícitamente de vuelta a número antes de pasárselo a un widget que espera un entero. Es un detalle fácil de pasar por alto y que causaría un error si se olvidara.

Los valores no se guardan automáticamente en cada cambio — hace falta presionar el botón "Guardar configuración", que llama a `db.guardar_configuracion_general(...)` pasando todos los valores actuales de una sola vez. Esta es una decisión de diseño deliberada: evita escribir a disco en cada tecla que el usuario presiona.

5.3 La tabla editable de puntos

`st.data_editor` es el widget que muestra la tabla de puntos como una hoja de cálculo editable directamente en el navegador. El parámetro `num_rows="dynamic"` permite agregar o quitar filas, y `column_config` define cómo se comporta cada columna (formato numérico, ancho del campo de texto, etc.).

Junto a la tabla hay dos botones relevantes: "Completar coordenadas desde dirección", que recorre las filas con una dirección escrita pero sin coordenadas, y llama a `geocodificar_direccion()` para cada una (con una pausa de un segundo entre llamadas, porque Nominatim pide no hacer más de una solicitud por segundo); y "Guardar puntos", que simplemente persiste la tabla actual en SQLite.

Nota: Después de geocodificar, el código llama a `st.rerun()`, que fuerza una nueva ejecución completa del script. Es necesario porque, de otra forma, el `data_editor` seguiría mostrando los valores viejos en pantalla aunque ya se hayan actualizado por detrás.

5.4 El botón "Calcular Ruta Óptima": el flujo completo

Este bloque conecta, en orden, todas las funciones explicadas en la sección 4. Vale la pena leerlo siguiendo el mismo camino que sigue el dato:

app.py — inicio del bloque de cálculo (resumido)

```
if st.button("Calcular Ruta Óptima", type="primary", use_container_width=True):
    db.guardar_puntos(tabla)
    puntos = tabla.dropna(subset=["Latitud", "Longitud"])
    if len(puntos) < 2:
        st.error("Necesitas al menos 2 puntos.")
        st.stop()

    DEPOT = (depot_lat, depot_lon)
    LOCATIONS = [DEPOT] + list(zip(puntos["Latitud"], puntos["Longitud"]))
    NOMBRES = ["DEPOT"] + puntos["Nombre"].tolist()
    PESOS = [0] + puntos["Peso (kg)"].tolist()
```

`tabla.dropna(subset=["Latitud", "Longitud"])` descarta cualquier fila que todavía no tenga coordenadas válidas (por ejemplo, una dirección que falló al geocodificar). `st.stop()` detiene la ejecución del script en ese punto exacto si no hay suficientes puntos — todo lo que está

después de esa línea, en esa ejecución, simplemente no corre.

La construcción `[DEPOT] + list(zip(...))` es clave: el depósito siempre queda en la posición 0 de la lista `LOCATIONS`, que es exactamente lo que `resolver_vrp` espera (recordemos que se le pasó 0 como índice del depósito al crear el `RoutingIndexManager`). `zip(...)` combina las columnas de latitud y longitud del `DataFrame` en pares (lat, lon), uno por cada punto.

El resto del bloque, simplificado, sigue esta secuencia:

- 1 `obtener_matriz_osrm(LOCATIONS)` — construye la matriz de distancias.
- 2 `resolver_vrp(distancias)` — obtiene el orden óptimo de visita.
- 3 Un bucle `for` llama a `obtener_ruta_osrm` tramo por tramo, para tener la geometría real que se va a dibujar en el mapa.
- 4 Otro bucle recorre la ruta calculando, parada por parada, la hora de llegada estimada (sumando tiempo de viaje según la distancia y velocidad configurada, más el tiempo fijo por parada) y el peso acumulado.
- 5 Todo el resultado se guarda en `st.session_state.resultados`, como un diccionario con todas las piezas que la sección de abajo necesita para dibujar.

Un detalle sutil pero importante en el cálculo de horarios:

app.py — cálculo de horarios (extracto)

```
hora_actual += timedelta(hours=(dist_m / 1000) / velocidad_kmh)
...
if not es_ultimo:
    hora_actual += timedelta(minutes=tiempo_parada)
```

Primero se suma el tiempo de viaje (distancia entre velocidad, convertido a horas), y después — solo si no es la última parada — se suma el tiempo fijo de parada. Esto evita sumar un tiempo de descarga innecesario después de la última entrega, justo antes de volver al depósito.

5.5 Mostrar resultados: métricas, tabla y mapa

Todo este bloque está envuelto en `if st.session_state.resultados`: — solo se dibuja si ya se calculó algo alguna vez en la sesión, lo que permite que el resultado siga visible aunque el usuario interactúe con otro control de la página.

Métricas con `st.columns`

`st.columns(4)` divide el ancho de la página en cuatro columnas iguales, y `.metric(...)` dibuja una tarjeta con un número grande y una etiqueta — el componente típico para mostrar indicadores clave de un vistazo.

El mapa con Folium

Folium es una librería de Python que genera mapas interactivos (basados en la librería de JavaScript Leaflet) sin que haga falta escribir una sola línea de JavaScript a mano. El patrón

general es siempre: crear el mapa, e ir agregándole elementos uno por uno.

app.py — construcción del mapa (extracto simplificado)

```
m = folium.Map(location=centro, zoom_start=12, tiles="OpenStreetMap")

for seg in r["segmentos"]:
    folium.PolyLine(seg["camino"], color="#E74C3C", weight=4, opacity=0.85).add_to(m)

folium.Marker(
    location=[lat, lon],
    popup="DEPOT",
    icon=folium.Icon(color="red", icon="home"),
).add_to(m)
```

`folium.PolyLine` dibuja una línea siguiendo la lista de coordenadas de cada tramo (la que devolvió `obtener_ruta_osrm`). `folium.Marker` pone un pin en el mapa; el depósito usa un ícono predefinido (casa), y cada parada usa un `DivIcon` con HTML/CSS a medida, para poder mostrar el número de orden de visita dentro de un círculo de color. El componente `st_folium` (de la librería `streamlit-folium`) es el puente que permite mostrar ese mapa de Folium dentro de la página de Streamlit.

Exportar: CSV y Google Maps

`st.download_button` genera un archivo en memoria (con `df.to_csv().encode("utf-8")`) y lo ofrece para descargar sin necesidad de guardar nada en el servidor. `st.link_button` simplemente abre una URL externa en una pestaña nueva — en este caso, la que construye `generar_link_google_maps`.

6. Conceptos clave para seguir aprendiendo

Streamlit: ejecución de arriba a abajo

El modelo mental más importante para trabajar con Streamlit es este: no hay eventos, hay re-ejecuciones completas. Cada vez que cambiás un control, Python corre el archivo entero otra vez, de la primera línea a la última. Lo único que persiste entre esas ejecuciones es `st.session_state`. Si algo se siente raro o inesperado en una app de Streamlit, casi siempre la explicación está en este punto.

APIs REST y manejo de errores

OSRM y Nominatim son APIs REST: servicios externos a los que se les pide información mandando una URL con parámetros, y que responden en formato JSON. El patrón que se repite en todo el proyecto — `try / except` capturando `Timeout` y `ConnectionError` por separado, con un plan B si falla — es la forma profesional de consumir servicios externos: nunca asumas que la red va a responder.

OR-Tools y los problemas de optimización combinatoria

El TSP y el VRP pertenecen a una familia de problemas donde la cantidad de soluciones posibles crece muchísimo más rápido que la cantidad de datos de entrada (crecimiento factorial). Por eso no se resuelven "probando todas las combinaciones" a partir de cierto tamaño, sino con heurísticas como las que vimos: construir una primera solución razonable rápido, y mejorarla iterativamente dentro de un tiempo límite.

SQLite como base de datos de archivo único

A diferencia de PostgreSQL o MySQL, SQLite no necesita un proceso de servidor corriendo aparte: toda la base de datos vive en un solo archivo (`routes.db`). Es ideal para aplicaciones de un solo usuario o equipos chicos, como esta. Cuando el uso concurrente crece (varios choferes consultando al mismo tiempo, por ejemplo), suele ser el momento de migrar a una base de datos con soporte real de concurrencia.

7. Glosario rápido

API / API REST — Un servicio al que otro programa le pide datos mandando una solicitud HTTP (normalmente una URL), y que responde en un formato estructurado como JSON.

Context manager — Un patrón de Python (la palabra clave `with`) que garantiza que un recurso (una conexión, un archivo) se cierre correctamente, pase lo que pase dentro del bloque.

DataFrame — La estructura de datos principal de la librería pandas: una tabla con filas y columnas, similar a una hoja de cálculo, pero manipulable con código.

Geocodificación — El proceso de convertir una dirección en texto a coordenadas geográficas (latitud y longitud).

Heurística — Una estrategia que encuentra una solución muy buena (aunque no garantice ser la mejor posible) en un tiempo razonable, usada cuando encontrar la solución perfecta sería demasiado costoso computacionalmente.

JSON — Un formato de texto para representar datos estructurados (objetos, listas, números, texto), el más usado para que las APIs intercambien información.

OR-Tools — Una librería de optimización de código abierto de Google, usada acá para resolver el problema de encontrar el mejor orden de visita entre varios puntos.

OSRM — Open Source Routing Machine: un servicio que calcula distancias y rutas reales siguiendo calles, a partir de datos de OpenStreetMap.

`session_state` — El diccionario especial de Streamlit que conserva valores entre una recarga del script y la siguiente, dentro de la misma sesión de navegador.

SQLite — Un motor de base de datos que guarda toda la información en un único archivo, sin necesitar un servidor de base de datos separado.

TSP / VRP — Travelling Salesman Problem y Vehicle Routing Problem: problemas clásicos de optimización que buscan el recorrido más corto que visite un conjunto de puntos, con uno o varios vehículos respectivamente.

Upsert — Una operación de base de datos que inserta una fila si no existe, o la actualiza si ya existe, en un solo paso.